

Tugas Kecil 1

Pengembangan Program Penyelesaian Permainan Queens LinkedIn berbasis Algoritma *Brute Force*

IF2211 Strategi Algoritma

Semester 2 Tahun 2025/2026



Disusun oleh:

Muhammad Aufar Rizqi Kusuma (13524061)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	<i>Problem Statement</i>	2
1.1	Deskripsi Awal	2
1.2	Permasalahan yang Harus Diselesaikan	2
2	Algoritma Penyelesaian	4
2.1	Pendekatan Umum	4
2.2	Solusi Teknis	4
3	Implementasi	7
3.1	Arsitektur Program	7
3.2	Direktori Program	7
3.2.1	IOHandler.java	8
3.2.2	BruteForce.java	10
3.2.3	ColorRegionExtractor.java	15
4	Eksperimen	22
4.1	Testing Input by TXT	22
4.2	Testing Input by Image	24
5	Penutup	27
6	Lampiran	28
6.1	Tabel CheckList	28
6.2	Pranala GitHub	28

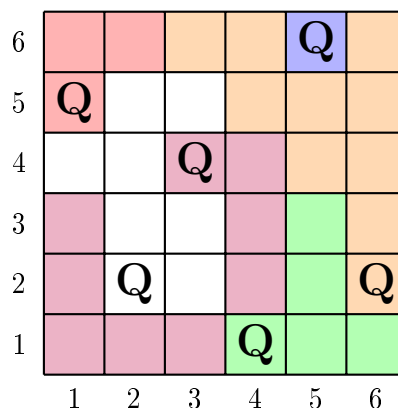
Bab 1

Problem Statement

1.1 Deskripsi Awal

Tugas Kecil 1 adalah tugas yang bertujuan untuk mengimplementasikan algoritma *brute force* dalam menyelesaikan permasalahan kombinatorial berbasis *constraint*, yaitu permainan Queens LinkedIn. Permainan ini merupakan variasi dari permasalahan klasik N-Queens, namun memiliki tambahan batasan berupa pembagian papan ke dalam wilayah warna tertentu serta aturan larangan *adjacency* antar queen.

Permainan dimainkan pada sebuah papan persegi berukuran $N \times N$ yang setiap selnya memiliki label huruf yang merepresentasikan suatu daerah warna. Pada kondisi awal, papan tidak berisi queen. Tugas utama program adalah menentukan konfigurasi penempatan queen yang memenuhi seluruh aturan permainan atau menyatakan bahwa solusi tidak ada.



Gambar 1.1: Ilustrasi permainan Queens dengan *constraint* baris, kolom, dan region

1.2 Permasalahan yang Harus Diselesaikan

Diberikan sebuah papan berwarna dalam bentuk matriks karakter, perlu ditentukan satu konfigurasi penempatan queen sehingga memenuhi seluruh *constraint* berikut.

- Setiap baris tepat mengandung satu queen.
- Setiap kolom tepat mengandung satu queen.
- Setiap daerah warna tepat mengandung satu queen.
- Tidak ada dua queen yang saling bersebelahan, baik secara horizontal, vertikal, maupun diagonal (*adjacent cell*).

Dengan demikian, solusi yang valid harus memenuhi *constraint* struktural (baris dan kolom), *constraint* regional (wilayah warna), serta *constraint* kedekatan (adjacency constraint). Permasalahan ini termasuk ke dalam kategori Constraint Satisfaction Problem (CSP), di mana solusi diperoleh dengan mengevaluasi kemungkinan konfigurasi terhadap seluruh batasan yang berlaku.

Bab 2

Algoritma Penyelesaian

2.1 Pendekatan Umum

Untuk menyelesaikan permasalahan *N-Queens* yang sudah dipaparkan di awal, terdapat berbagai metode yang dapat diterapkan. Di tugas kecil ini, kita akan menggunakan algoritma yang paling *naif/intuitif*, yaitu algoritma *Brute Force*. Pada dasarnya, dengan menggunakan algoritma ini, kita akan menghasilkan seluruh *state* yang mungkin dari peletakan bidak ratu pada tiap-tiap kolom. *State* yang dimaksud nantinya akan berupa konfigurasi peletakan bidak ratu pada tiap-tiap kolom. Jika *state* tersebut mematuhi seluruh *constraint* yang ada, maka *state* tersebut adalah solusi dan akan dimasukkan ke dalam himpunan solusi.

Perhatikan bahwa untuk papan catur berukuran $N \times N$, maka tiap kolom akan memiliki N buah cara penempatan dan jumlah kolomnya adalah sebanyak N buah. Sehingga, secara teoretis, banyak solusi yang harus dicek adalah sebanyak N^N buah. Tentunya, pendekatan ini bukanlah pendekatan paling optimal, malah dapat dikatakan bahwa pendekatan ini merupakan paling boros karena mengharuskan program untuk mengecek seluruh *state* yang mungkin. Namun, ini penting sebagai langkah awal dalam memahami strategi dalam menyelesaikan sebuah permasalahan.

2.2 Solusi Teknis

Dalam membangkitkan seluruh *state* yang mungkin, maka *state* akan dinyatakan sebagai sebuah *list of int* bernama `queens`, dimana `queens[col] = row` menyatakan baris ke-`row` dari bidak `queens` pada kolom ke-`col`. Lebih lanjut, prosedur pembangkitan akan dilakukan dengan mengatur `queens[i] = 0,  $\forall i \in [0, N - 1]$` . Lalu, secara berangsur-angsur, bidak `queens` terujung akan ditambahkan secara *incremental*. Jika bidak terujung tersebut nilainya sudah mencapai $N - 1$, maka bidang ujung akan diatur menjadi 0 kembali dan bidak ratu sebelum ujung akan diatur menjadi 1. Kemudian, bidak ujung kembali di-*increment* hingga, mencapai $N - 1$ sehingga nanti bidak ratu sebelum ujung akan diatur menjadi 2. Begitu seterusnya hingga *state* akhirnya adalah ketika `queens = [N-1, N-1, ..., N-1]`. Dalam program, proses ini akan ditangani oleh metode yang bernama `nextState()`.

Algorithm 1 Generate Next State Configuration `nextState()`

Require: Integer n , array `queens` $[0 \dots n - 1]$

Ensure: **true** if next configuration exists; otherwise **false**

```
1: for  $i \leftarrow n - 1$  downto 0 do
2:   if queens $[i] < n - 1$  then
3:     queens $[i] \leftarrow$  queens $[i] + 1$ 
4:     return true
5:   else
6:     queens $[i] \leftarrow 0$ 
7:   end if
8: end for
9: return false
```

Gambar 2.1: *Pseudocode* untuk algoritma pembangkitan *state* yang mungkin.

Selanjutnya, untuk mengecek apakah *state* yang telah dihasilkan tersebut mematuhi *constraint* yang sudah dibuat, maka perlu ada sebuah fungsi yang dapat mengecek apakah *state* tersebut valid atau tidak. Perhatikan bahwa karena kita menyimpan *state* dalam bentuk *list of integer* bernama `queens`, maka kita sudah menjamin bahwa bidak ratu akan berada pada kolom yang berbeda-beda. Sehingga, kita hanya perlu mengecek apakah

- Apakah tiap bidak ratu berada pada kolom yang berbeda-beda?
- Apakah pada tiap daerah hanya ditempati tepat oleh satu bidak ratu saja?
- Apakah tidak elemen diagonal tetangga antar tiap ratu?

Hal inilah yang akan dicek setiap kita mengenumerasi sebuah *state*. Jika *state* tersebut valid, maka *state* tersebut akan dimasukkan ke dalam himpunan solusi. Dalam program, pengecekan ini akan dilakukan oleh metode bernama `isValid()`.

Algorithm 2 Validation of Queen Configuration with Region and Adjacency Constraints

Require: Integer n , array $queens[0 \dots n - 1]$, matrix $board[0 \dots n - 1][0 \dots n - 1]$
Ensure: **true** if configuration is valid; otherwise **false**

```
1: for  $i \leftarrow 0$  to  $n - 1$  do
2:   for  $j \leftarrow i + 1$  to  $n - 1$  do           ▷ Check same row or same region/color
3:     if  $queens[i] = queens[j]$  or  $board[queens[i]][i] = board[queens[j]][j]$  then
4:       return false
5:     end if                                   ▷ Check adjacency constraint
6:     if  $|i - j| = 1$  and  $|queens[i] - queens[j]| \leq 1$  then
7:       return false
8:     end if
9:   end for
10: end for
11: return true
```

Gambar 2.2: *Pseudocode* pengecekan kesahihan *state* bidak ratu.

Dengan kedua metode ini, kita sudah bisa melakukan penelusuran terhadap semua solusi yang mungkin dan menghasilkan himpunan solusi yang mungkin dengan konfigurasi awal papan catur yang dapat diatur sesuai dengan keinginan pengguna.

Algorithm 3 Brute Force Search for Modified *N-Queens* Solution

Require: Integer n , matrix $board[0 \dots n - 1][0 \dots n - 1]$
Ensure: A valid queen configuration if exists; otherwise **no solution**

```
1: Initialize  $queens[0 \dots n - 1] \leftarrow 0$ 
2:  $found \leftarrow \text{false}$ 
3: repeat
4:   if ISVALID( $board$ ) then
5:      $found \leftarrow \text{true}$ 
6:     return  $queens$ 
7:   end if
8: until NEXTSTATE = false
9: return no solution
```

Gambar 2.3: *Pseudocode* penyelesaian masalah *N-Queens* termodifikasi dengan pendekatan *brute force*.

Bab 3

Implementasi

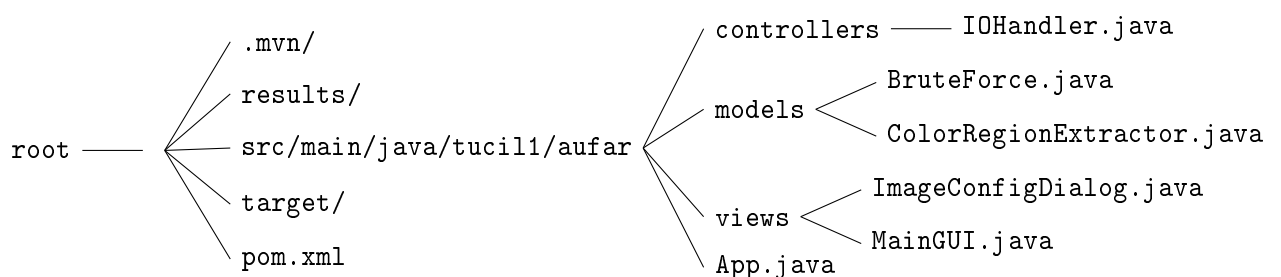
3.1 Arsitektur Program

Program yang dibuat menggunakan arsitektur MVC, dimana program akan dibagi menjadi beberapa layer, yaitu layer *Model*, *View*, dan *Controller*. Layer *model* akan bertanggung jawab untuk menangani logika dari program. Dalam konteks ini, model akan bertanggungjawab untuk menangani algoritma *brute force* dalam menyelesaikan program dan menangani pula *color extractor* yang akan digunakan untuk bonus *input by image*.

Layer *view* akan bertanggung jawab terhadap antarmuka (GUI) program. Nantinya, layer ini akan menangani masukan dari pengguna sembari mengeluarkan output berdasarkan hasil dari algoritma yang diterapkan pada layer model.

Layer *controller* akan menjadi *intermediate layer* yang menghubungkan antarmuka program dengan logika dari program. Perlu diperhatikan bahwa keluaran dari logika tidak bisa langsung ditampilkan ke dalam GUI. Perlu adanya pemrosesan lebih lanjut agar keluaran yang dihasilkan sesuai dengan format yang ditentukan oleh GUI. Adanya *controller* akan menangani permasalahan tersebut dan memastikan hubungan antar *view* dan *model* dapat berjalan dengan baik.

3.2 Direktori Program



Gambar 3.1: Struktur direktori program

3.2.1 IOHandler.java

```
1
2 package tucil1.aufar.controllers;
3
4 import java.io.BufferedReader;
5 import java.io.BufferedWriter;
6 import java.io.File;
7 import java.io.FileReader;
8 import java.io.FileWriter;
9 import java.io.IOException;
10 import java.util.ArrayList;
11 import java.util.List;
12
13 public class IOHandler {
14
15     private char[][] board;
16     private int N;
17     private String errorMessage = "";
18
19     public boolean readFile(String filename) {
20         List<char[]> tempBoard = new ArrayList<>();
21         File file = new File(filename);
22
23         if (!file.exists()) {
24             file = new File("test/" + filename);
25         }
26
27         if (!file.exists()) {
28             file = new File("../test/" + filename);
29         }
30
31         if (!file.exists()) {
32             errorMessage = "File tidak ditemukan: " +
33                 filename;
34             return false;
35         }
36
37         try (BufferedReader br = new BufferedReader(
38             new FileReader(file))) {
39
40             String line;
41
42             while ((line = br.readLine()) != null) {
43                 line = line.trim();
44                 if (!line.isEmpty()) {
45                     tempBoard.add(line.toCharArray());
46                 }
47             }
48         }
49     }
50 }
```

```
47
48     } catch (IOException e) {
49         errorMessage = "Gagal membaca file: "
50         + e.getMessage();
51         return false;
52     }
53
54     if (tempBoard.isEmpty()) {
55         errorMessage = "File kosong.";
56         return false;
57     }
58
59     N = tempBoard.size();
60     board = new char[N][];
61
62     for (int i = 0; i < N; i++) {
63         board[i] = tempBoard.get(i);
64     }
65
66     if (!isSquare()) {
67         errorMessage =
68             "Papan tidak berbentuk persegi (N x N).";
69         return false;
70     }
71
72     return isAllAlphabet();
73 }
74
75 private boolean isSquare() {
76     for (int i = 0; i < N; i++) {
77         if (board[i].length != N) {
78             return false;
79         }
80     }
81     return true;
82 }
83
84 private boolean isAllAlphabet() {
85     for (int i = 0; i < N; i++) {
86         for (int j = 0; j < N; j++) {
87             char c = board[i][j];
88             if (!Character.isLetter(c)) {
89                 errorMessage =
90                     "Karakter tidak valid '" + c +
91                     "' pada baris " + (i+1) +
92                     ", kolom " + (j+1) +
93                     ". Hanya alfabet (A-Z, a-z) "
94                     + "yang diperbolehkan.";
95                 return false;
96             }
97         }
98     }
99     return true;
100 }
```

```
96         }
97     }
98 }
99     return true;
100 }
101
102 public char[][] getBoard() {
103     return board;
104 }
105
106 public int getSize() {
107     return N;
108 }
109
110 public String getErrorMessage() {
111     return errorMessage;
112 }
113
114 public boolean writeToFile(String filename,
115 String content) {
116
117     File outputDir = new File("results");
118     if (!outputDir.exists()) {
119         outputDir.mkdirs();
120     }
121
122     File file = new File(outputDir, filename);
123
124     try (BufferedWriter bw =
125         new BufferedWriter(
126             new FileWriter(file))) {
127
128         bw.write(content);
129         return true;
130
131     } catch (IOException e) {
132         errorMessage =
133             "Gagal menulis file: "
134             + e.getMessage();
135         return false;
136     }
137 }
138 }
```

Listing 3.1: Implementasi Kelas IOHandler untuk Pembacaan dan Penulisan File

3.2.2 BruteForce.java

```
2 package tucil1.aufar.models;
3
4 import java.util.ArrayList;
5 import java.util.List;
6 import java.util.function.BiConsumer;
7
8 public class BruteForce {
9     int n;
10    int[] queens;
11    List<String> solutions;
12    List<int[]> solutionQueens;
13
14    // Stats
15    private long searchTimeMs;
16    private long casesCheckedTotal;
17
18    // Callbacks for GUI visualization
19    private VisualizationCallback visualizationCallback;
20    private BiConsumer<String, Integer> solutionCallback;
21
22    @FunctionalInterface
23    public interface VisualizationCallback {
24        void onUpdate(int[] queens, long casesChecked,
25            double progress);
26    }
27
28    public BruteForce(int n) {
29        this.n = n;
30        this.queens = new int[n];
31        this.solutions = new ArrayList<>();
32        this.solutionQueens = new ArrayList<>();
33    }
34
35    public void setVisualizationCallback(
36        VisualizationCallback callback) {
37        this.visualizationCallback = callback;
38    }
39
40    public void setSolutionCallback(
41        BiConsumer<String, Integer> callback) {
42        this.solutionCallback = callback;
43    }
44
45    private boolean nextState() {
46        for (int i = n-1; i >= 0; i--) {
47            if (queens[i] < n-1) {
48                queens[i]++;
49                return true;
50            } else {
```

```
50         queens[i] = 0;
51     }
52 }
53 return false;
54 }
55
56 private boolean isValid(char[][] board) {
57     for (int i = 0; i < n; i++) {
58         for (int j = i+1; j < n; j++) {
59
60             // Same row or same region
61             if (this.queens[i] == this.queens[j] ||
62                 board[queens[i]][i] ==
63                 board[queens[j]][j]) {
64                 return false;
65             }
66
67             // Adjacent constraint
68             if (Math.abs(i - j) == 1 &&
69                 Math.abs(queens[i] -
70                     queens[j]) <= 1) {
71                 return false;
72             }
73         }
74     }
75     return true;
76 }
77
78 private String getSolutionString(char[][] board) {
79     StringBuilder sb = new StringBuilder();
80
81     for (int row = 0; row < n; row++) {
82         for (int col = 0; col < n; col++) {
83             if (queens[col] == row) {
84                 sb.append('#');
85             } else {
86                 sb.append(board[row][col]);
87             }
88         }
89         sb.append("\n");
90     }
91     return sb.toString();
92 }
93
94 public String solve(char[][] board) {
95
96     long startTime = System.nanoTime();
97     long casesChecked = 0;
98 }
```

```
99         solutions.clear();
100        solutionQueens.clear();
101        StringBuilder result =
102        new StringBuilder();
103
104        long totalCase =
105        (long) Math.pow(n, n);
106
107        int[] liveUpdate =
108        {10,20,30,40,50,60,70,80,90};
109
110        int update = 0;
111        long visualizationInterval =
112        Math.max(1, totalCase / 100);
113
114        do {
115            casesChecked++;
116
117            if (visualizationCallback != null &&
118            casesChecked %
119            visualizationInterval == 0) {
120
121                double progress =
122                (casesChecked * 100.0)
123                / totalCase;
124
125                visualizationCallback.onUpdate(
126                queens.clone(),
127                casesChecked,
128                progress);
129            }
130
131            if (update < liveUpdate.length) {
132                if ((casesChecked * 100
133                / totalCase)
134                > liveUpdate[update]) {
135
136                    System.out.println(
137                    "Update Ke - "
138                    + (update+1));
139
140                    result.append("Update Ke -")
141                    .append(update+1)
142                    .append("\n");
143
144                    String boardState =
145                    getSolutionString(board);
146
147                    System.out.print(boardState);
```

```
148         result.append(boardState)
149         .append("\n");
150
151         update++;
152     }
153 }
154
155 if (isValid(board)) {
156     String solution =
157         getSolutionString(board);
158
159     solutions.add(solution);
160     solutionQueens.add(
161         queens.clone());
162
163     if (solutionCallback != null) {
164         solutionCallback.accept(
165             solution,
166             solutions.size());
167     }
168 }
169
170 } while (nextState()
171 && solutions.size() < 1);
172
173 long endTime = System.nanoTime();
174 searchTimeMs =
175 (endTime - startTime) / 1_000_000;
176
177 casesCheckedTotal = casesChecked;
178
179 if (solutions.size() < 1) {
180     System.out.println(
181         "Tidak Ada Solusi!");
182     result.append(
183         "Tidak Ada Solusi!");
184 } else {
185     System.out.println(
186         "\nSOLUSI AKHIR");
187
188     result.append(
189         "\nSOLUSI AKHIR\n");
190
191     System.out.print(
192         solutions.get(0) + "\n");
193
194     result.append(
195         solutions.get(0))
196         .append("\n");
```

```
197         }
198
199         String stats =
200             "Waktu pencarian: "
201             + searchTimeMs + " ms\n"
202             + "Banyak kasus yang ditinjau: "
203             + casesChecked + " kasus\n";
204
205         System.out.println(stats);
206         result.append(stats);
207
208         return result.toString();
209     }
210
211     public int getSolutionCount() {
212         return solutions.size();
213     }
214
215     public long getSearchTimeMs() {
216         return searchTimeMs;
217     }
218
219     public long getCasesChecked() {
220         return casesCheckedTotal;
221     }
222
223     public int[] getFirstSolutionQueens() {
224         if (solutionQueens.isEmpty()) {
225             return null;
226         }
227         return solutionQueens.get(0);
228     }
229
230     public List<int[]> getAllSolutionQueens() {
231         return solutionQueens;
232     }
233 }
```

Listing 3.2: Implementasi Algoritma Brute Force untuk LinkedIn Queens Variant

3.2.3 ColorRegionExtractor.java

```
1
2     package tucil1.aufar.models;
3
4     import java.util.ArrayList;
5     import java.util.List;
6
7     import javafx.scene.image.Image;
```

```
8 import javafx.scene.image.PixelReader;
9 import javafx.scene.paint.Color;
10
11 /**
12  * Extract board regions from images using HSB color
13  * analysis.
14  */
15 public class ColorRegionExtractor {
16
17     private char[][] resultBoard;
18     private int regionCount;
19
20     private double satThreshold = 0.20;
21     private double brightThreshold = 0.18;
22
23     public char[][] extractBoard(Image image,
24     int gridX, int gridY,
25     int gridWidth,
26     int gridHeight,
27     int boardSize) {
28
29         PixelReader reader =
30             image.getPixelReader();
31
32         int cellW = gridWidth / boardSize;
33         int cellH = gridHeight / boardSize;
34
35         resultBoard =
36             new char[boardSize][boardSize];
37
38         List<double[]> knownColors =
39             new ArrayList<>();
40
41         char nextRegion = 'A';
42
43         int imgW = (int) image.getWidth();
44         int imgH = (int) image.getHeight();
45
46         for (int row = 0;
47             row < boardSize; row++) {
48
49             for (int col = 0;
50                 col < boardSize; col++) {
51
52                 Color cellColor =
53                     sampleCellColor(
54                         reader,
55                         gridX + col * cellW,
```

```
56         cellW, cellH,  
57         imgW, imgH);  
58  
59         double[] hsb =  
60         colorToHSB(cellColor);  
61  
62         int matchedRegion =  
63         findMatchingColor(  
64         knownColors, hsb);  
65  
66         if (matchedRegion == -1) {  
67             knownColors.add(hsb);  
68             resultBoard[row][col] =  
69             nextRegion++;  
70         } else {  
71             resultBoard[row][col] =  
72             (char) ('A'  
73             + matchedRegion);  
74         }  
75     }  
76 }  
77  
78     regionCount =  
79     knownColors.size();  
80  
81     return resultBoard;  
82 }  
83  
84 public Color sampleCellColor(  
85 PixelReader reader,  
86 int cellX, int cellY,  
87 int cellW, int cellH,  
88 int imgW, int imgH) {  
89  
90     int[][] offsets = {  
91         {cellW/2, cellH/2},  
92         {cellW/3, cellH/3},  
93         {2*cellW/3, cellH/3},  
94         {cellW/3, 2*cellH/3},  
95         {2*cellW/3, 2*cellH/3}  
96     };  
97  
98     double totalR = 0,  
99     totalG = 0,  
100    totalB = 0;  
101  
102    int count = 0;  
103  
104    for (int[] offset : offsets) {
```

```
105
106         int x = Math.min(
107             cellX + offset[0],
108             imgW - 1);
109
110         int y = Math.min(
111             cellY + offset[1],
112             imgH - 1);
113
114         x = Math.max(0, x);
115         y = Math.max(0, y);
116
117         Color c =
118             reader.getColor(x, y);
119
120         totalR += c.getRed();
121         totalG += c.getGreen();
122         totalB += c.getBlue();
123         count++;
124     }
125
126     return Color.color(
127         totalR / count,
128         totalG / count,
129         totalB / count);
130 }
131
132 public double[] colorToHSB(Color color) {
133
134     double r = color.getRed();
135     double g = color.getGreen();
136     double b = color.getBlue();
137
138     double max =
139         Math.max(r,
140             Math.max(g, b));
141
142     double min =
143         Math.min(r,
144             Math.min(g, b));
145
146     double delta = max - min;
147
148     double hue = 0;
149     double saturation =
150         (max == 0)
151         ? 0
152         : delta / max;
153 }
```

```
154         double brightness = max;
155
156         if (delta != 0) {
157             if (max == r) {
158                 hue = 60 *
159                     (((g - b)
160                      / delta) % 6);
161             } else if (max == g) {
162                 hue = 60 *
163                     (((b - r)
164                      / delta) + 2);
165             } else {
166                 hue = 60 *
167                     (((r - g)
168                      / delta) + 4);
169             }
170         }
171
172         if (hue < 0) hue += 360;
173
174         return new double[]{
175             hue,
176             saturation,
177             brightness};
178     }
179
180     public int findMatchingColor(
181     List<double[]> knownColors,
182     double[] hsb) {
183
184         for (int i = 0;
185             i < knownColors.size();
186             i++) {
187
188             double[] known =
189                 knownColors.get(i);
190
191             double hueDiff =
192                 Math.abs(hsb[0]
193                     - known[0]);
194
195             if (hueDiff > 180)
196                 hueDiff =
197                     360 - hueDiff;
198
199             double satDiff =
200                 Math.abs(hsb[1]
201                     - known[1]);
202
```

```
203         double brightDiff =
204             Math.abs(hsb[2]
205                 - known[2]);
206
207         double hueThreshold =
208             getHueThreshold(
209                 hsb[0], known[0]);
210
211         if (hsb[1] < 0.15
212             && known[1] < 0.15) {
213
214             if (brightDiff < 0.12)
215                 return i;
216
217             } else if (
218                 hueDiff < hueThreshold
219                 && satDiff < satThreshold
220                 && brightDiff < brightThreshold) {
221
222                 return i;
223             }
224     }
225
226     return -1;
227 }
228
229 public double getHueThreshold(
230     double hue1,
231     double hue2) {
232
233     boolean critical1 =
234         (hue1 >= 0 && hue1 <= 60)
235         || (hue1 >= 330
236             && hue1 <= 360);
237
238     boolean critical2 =
239         (hue2 >= 0 && hue2 <= 60)
240         || (hue2 >= 330
241             && hue2 <= 360);
242
243     if (critical1 && critical2)
244         return 12;
245
246     boolean green1 =
247         (hue1 >= 90
248             && hue1 <= 180);
249
250     boolean green2 =
251         (hue2 >= 90
```

```
252         && hue2 <= 180);  
253  
254         if (green1 && green2)  
255             return 15;  
256  
257         return 20;  
258     }  
259  
260     public char[][] getResultBoard() {  
261         return resultBoard;  
262     }  
263  
264     public int getRegionCount() {  
265         return regionCount;  
266     }  
267  
268     public void setSaturationThreshold(  
269         double threshold) {  
270         this.satThreshold =  
271             threshold;  
272     }  
273  
274     public void setBrightnessThreshold(  
275         double threshold) {  
276         this.brightThreshold =  
277             threshold;  
278     }  
279 }
```

Listing 3.3: Implementasi ColorRegionExtractor untuk Ekstraksi Region Berbasis Warna (HSB)

Bab 4

Eksperimen

4.1 Testing Input by TXT

Tabel 4.1: Hasil Pengujian Algoritma Brute Force dengan masukan berformat TXT

No	Test Case	Hasil
1		

Lanjut ke halaman berikutnya

No	Test Case	Hasil
2		
3		

Lanjut ke halaman berikutnya

No	Test Case	Hasil
4		Solusi (Brute-force): <pre> SOLUSI AKHIR #AABBB AAA#BB C#CDDD CCCD#D EE#FFF EEEEF# </pre> Waktu pencarian: 1 ms Banyak kasus yang ditinjau: 3516 kasus
5		Tidak Ada Solusi! Waktu pencarian: 346 ms Banyak kasus yang ditinjau: 16777216 kasus

4.2 Testing Input by Image

Tabel 4.2: Hasil Pengujian Algoritma Brute Force dengan masukan berformat Image

No	Test Case	Hasil
1		
2		
3		
4		

Lanjut ke halaman berikutnya

No	Test Case	Hasil
5		

Bab 5

Penutup

Sebagai penutup, tugas kecil 1 ini menantang kami untuk bisa berpikir untuk menyelesaikan masalah dengan metode paling natural yang dimiliki oleh manusia. Metode yang dimaksud adalah metode *brute force*, dimana akan diselidiki seluruh ruang keadaan yang mungkin dari suatu permasalahan dan mengecek apakah keadaan tersebut memenuhi aturan-aturan yang berlaku. Paradigma berpikir ini kemudian dituangkan dalam menyelesaikan permasalahan *N-Queens* termodifikasi pada *web LinkedIn*. Menurut saya, ini merupakan langkah penting dalam mempelajari strategi dalam menyelesaikan sebuah permasalahan. Harapannya, seiring tugas-tugas mendatang, saya dapat lebih mematangkan kemampuan saya dalam menyelesaikan permasalahan. **Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.**

Bandung, 15 Februari 2026

Muhammad Aufar Rizqi Kusuma
13524061

Bab 6

Lampiran

6.1 Tabel CheckList

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	

6.2 Pranala GitHub

tucil1-queens-game-simulation